

Entrada, cálculo e print com casas decimais

Programa completo que lê dois valores, calcula resultados e controla exatamente a apresentação decimal.

Entrada: dois números. Saída: soma, média e produto com 2 casas.

Tempo $O(1)$ · memória $O(1)$

EXEMPLO DE CÓDIGO

solucao.py

```
1 import sys
2
3
4 def main():
5     # split separa a linha; map converte cada parte para float.
6     a, b = map(float, input().split())
7
8     # Os cálculos usam a precisão original dos valores.
9     soma = a + b
10    media = soma / 2.0
11    produto = a * b
12
13    # {:.2f} arredonda apenas na apresentação e sempre mostra 2 casas.
14    print(f"Soma = {soma:.2f}")
15    print(f"Media = {media:.2f}")
16    print(f"Produto = {produto:.2f}")
17
18    # O juiz normalmente espera uma quebra após a última linha.
19    # print já acrescenta essa quebra automaticamente.
20
21
22 if __name__ == "__main__":
23     main()
```

TÉCNICA

`input()` devolve texto; a conversão define quais operações podem ser feitas. A f-string separa cálculo de formatação.

ARMADILHA

Não arredonde antes de terminar os cálculos. Confira maiúsculas, rótulos, espaços e número de casas exigidos.

T casos com if, elif e else

Programa completo que processa uma quantidade declarada de notas e classifica cada caso por intervalos.

Entrada: T e uma nota por caso. Saída: classificação por linha.

Tempo $O(T)$ · memória $O(1)$

EXEMPLO DE CÓDIGO

solucao.py

```
1 def classificar(nota):
2     # A ordem impede que um intervalo capture outro.
3     if 90 <= nota <= 100:
4         return "A"
5     elif 70 <= nota < 90:
6         return "B"
7     elif 60 <= nota < 70:
8         return "C"
9     elif 0 <= nota < 60:
10        return "D"
11    else:
12        return "nota invalida"
13
14
15 def main():
16     # A primeira linha informa quantos casos virão.
17     quantidade = int(input())
18
19     for caso in range(1, quantidade + 1):
20         nota = int(input())
21         conceito = classificar(nota)
22         print(f"Caso {caso}: {conceito}")
23
24
25 if __name__ == "__main__":
26     main()
```

TÉCNICA

for range(1, T + 1) produz casos numerados de 1 a T. A função devolve a decisão; o main cuida do protocolo.

ARMADILHA

Teste 0, 59, 60, 69, 70, 89, 90 e 100. Bordas inclusivas são fonte frequente de WA.

while, sentinela, break e continue

Programa completo que lê valores até zero, ignora negativos e acumula somente os positivos válidos.

Entrada: inteiros até aparecer 0. Saída: quantidade e soma.

Tempo $O(n)$ · memória $O(1)$

EXEMPLO DE CÓDIGO

solucao.py

```
1 def main():
2     quantidade = 0
3     soma = 0
4
5     while True:
6         valor = int(input())
7
8         # Zero encerra a entrada e não é processado.
9         if valor == 0:
10             break
11
12         # Negativos são ignorados nesta regra.
13         if valor < 0:
14             continue
15
16         # Só valores positivos chegam até aqui.
17         quantidade += 1
18         soma += valor
19
20     print(f"Quantidade: {quantidade}")
21     print(f"Soma: {soma}")
22
23
24 if __name__ == "__main__":
25     main()
```

TÉCNICA

break encerra o laço inteiro. **continue** pula apenas o restante da repetição atual e volta à leitura.

ARMADILHA

Em laços com índice, atualize o estado antes de um possível **continue**; caso contrário o mesmo estado pode repetir para sempre.

Leitura até EOF preservando a linha

Programa completo que recebe frases até o fim do arquivo e conta letras, dígitos e espaços em cada uma.

Entrada: número desconhecido de linhas. Saída: três contagens ...

Tempo $O(\text{total de caracteres})$ · memória $O(1)$ extra

EXEMPLO DE CÓDIGO

solucao.py

```
1 import sys
2
3
4 def analisar(texto):
5     letras = 0
6     digitos = 0
7     espacos = 0
8
9     for caractere in texto:
10         if caractere.isalpha():
11             letras += 1
12         elif caractere.isdigit():
13             digitos += 1
14         elif caractere == " ":
15             espacos += 1
16
17     return letras, digitos, espacos
18
19
20 def main():
21     # 0 for termina naturalmente quando o arquivo acaba.
22     for linha in sys.stdin:
23         # Remove apenas a quebra, preservando outros espaços.
24         texto = linha.rstrip("\n")
25         l, d, e = analisar(texto)
26         print(l, d, e)
27
28
29 if __name__ == "__main__":
30     main()
```

TÉCNICA

EOF não é um valor da entrada: é a ausência de outra linha. `for linha in sys.stdin` expressa esse formato diretamente.

ARMADILHA

`strip()` também remove espaços significativos das pontas. Use `rstrip("\n")` quando o conteúdo completo da linha importa.

Entrada e saída rápidas em bloco

Programa completo para muitos tokens: lê tudo como bytes, consome com iterador e escreve todas as respostas de uma vez.

Entrada: T listas numéricas. Saída: soma de cada lista.

Tempo $O(\text{total de tokens})$ · memória $O(\text{total da entrada})$

EXEMPLO DE CÓDIGO

solucao.py

```
1 import sys
2
3
4 def main():
5     # read().split() é rápido quando espaços e linhas são equivalentes.
6     dados = iter(sys.stdin.buffer.read().split())
7     quantidade_de_casos = int(next(dados))
8     respostas = []
9
10    for _ in range(quantidade_de_casos):
11        tamanho = int(next(dados))
12        soma = 0
13
14        for _ in range(tamanho):
15            soma += int(next(dados))
16
17        # Guardamos texto para uma única escrita no final.
18        respostas.append(str(soma))
19
20    sys.stdout.write("\n".join(respostas) + "\n")
21
22
23 if __name__ == "__main__":
24     main()
```

TÉCNICA

O iterador consome exatamente os tokens do caso atual. `join` reduz milhares de chamadas de escrita.

ARMADILHA

Essa técnica perde a distinção entre linhas. Não a use quando frases, linhas vazias ou espaços internos fazem parte do dado.

for, range, enumerate e matriz

Programa completo que lê uma matriz, percorre valores e índices e encontra a posição do maior elemento.

Entrada: linhas, colunas e matriz. Saída: soma e posição do má...

Tempo $O(\text{linhas} \cdot \text{colunas})$ · memória $O(\text{linhas} \cdot \text{colunas})$

EXEMPLO DE CÓDIGO

solucao.py

```
1 def main():
2     linhas, colunas = map(int, input().split())
3     matriz = [
4         list(map(int, input().split()))
5         for _ in range(linhas)
6     ]
7
8     soma = 0
9     maior = matriz[0][0]
10    melhor_linha = 0
11    melhor_coluna = 0
12
13    for i, linha in enumerate(matriz):
14        for j, valor in enumerate(linha):
15            soma += valor
16
17            if valor > maior:
18                maior = valor
19                melhor_linha = i
20                melhor_coluna = j
21
22    print(f"Soma: {soma}")
23    print(f"Maior: {maior}")
24    print(f"Posicao: {melhor_linha + 1} {melhor_coluna + 1}")
25
26
27 if __name__ == "__main__":
28     main()
```

TÉCNICA

enumerate entrega posição e valor. Índices internos começam em 0; a saída pode exigir conversão para base 1.

ARMADILHA

Não assumam matriz não vazia sem conferir as restrições. Cada linha também precisa conter a quantidade declarada de colunas.

Varredura de lista com melhor resposta

Programa completo que encontra o palpite mais próximo e preserva o primeiro participante quando há empate.

Entrada: T casos, alvo e palpites. Saída: posição vencedora.

Tempo $O(\text{total de palpites})$ · memória $O(n)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 N = int(input().strip())
2
3 for _ in range(N):
4     QT, S = [int(x) for x in input().strip().split(' ')]
5
6     palpites = [int(x) for x in input().strip().split(' ')]
7
8     # Começamos considerando o primeiro aluno como o melhor.
9     melhor_palpite = 0
10    menor_diferenca = abs(S - palpites[0])
11
12    # Só trocamos em uma diferença estritamente menor; assim, empates mantêm
13    # o primeiro palpite, como exige o enunciado.
14    for i in range(1, QT):
15        if(menor_diferenca > abs(S - palpites[i])):
16            melhor_palpite = i
17            menor_diferenca = abs(S - palpites[i])
18
19    print(melhor_palpite + 1)
```

TÉCNICA

A melhor resposta começa no primeiro elemento. A comparação estrita atualiza apenas quando a nova distância é realmente menor.

ARMADILHA

Trocar $\leq$ por $<$ altera o desempate. A posição impressa é índice + 1.

Funções, math e estatística básica

Programa completo que decompõe cálculos em funções e imprime distância, média e mediana.

Entrada: dois pontos e uma lista. Saída: medidas com 3 casas.

Tempo $O(n \log n)$ pela ordenação · memória $O(n)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 import math
2
3
4 def distancia(x1, y1, x2, y2):
5     # hypot calcula  $\sqrt{dx^2 + dy^2}$ .
6     return math.hypot(x1 - x2, y1 - y2)
7
8
9 def media(valores):
10     return sum(valores) / len(valores)
11
12
13 def mediana(valores):
14     ordenados = sorted(valores)
15     n = len(ordenados)
16     meio = n // 2
17
18     if n % 2 == 1:
19         return ordenados[meio]
20     return (ordenados[meio - 1] + ordenados[meio]) / 2
21
22
23 def main():
24     x1, y1, x2, y2 = map(float, input().split())
25     n = int(input())
26     valores = list(map(float, input().split()))
27     assert len(valores) == n
28
29     print(f"Distancia: {distancia(x1, y1, x2, y2):.3f}")
30     print(f"Media: {media(valores):.3f}")
31     print(f"Mediana: {mediana(valores):.3f}")
32
33
34 if __name__ == "__main__":
35     main()
```

TÉCNICA

Funções separam contratos pequenos: parâmetros entram, return devolve o resultado e print fica no programa principal.

ARMADILHA

`math.sin` recebe radianos. Quando o enunciado fornece graus, converta com `math.radians(angulo)`.

Dicionário e set para frequências

Programa completo que conta palavras, remove duplicatas e responde consultas de ocorrência.

Entrada: palavras e consultas. Saída: frequência ou zero.

Tempo $O(n+q)$ médio · memória $O(n)$

EXEMPLO DE CÓDIGO

solucao.py

```
1 def main():
2     n, q = map(int, input().split())
3     palavras = input().split()
4     assert len(palavras) == n
5
6     frequencia = {}
7     unicas = set()
8
9     for palavra in palavras:
10         frequencia[palavra] = frequencia.get(palavra, 0) + 1
11         unicas.add(palavra)
12
13     print(f"Distintas: {len(unicas)}")
14
15     for _ in range(q):
16         consulta = input().strip()
17         print(frequencia.get(consulta, 0))
18
19
20 if __name__ == "__main__":
21     main()
```

TÉCNICA

O dicionário associa palavra à contagem. O set responde pertencimento e quantidade de valores distintos.

ARMADILHA

set não preserva duplicatas. A ordem de iteração não deve ser usada como regra de saída, salvo garantia explícita.

Ordenação com múltiplos critérios

Programa completo que ordena registros por peso decrescente, idade, altura e nome crescentes.

Entrada: cenários de renas. Saída: prefixo do ranking.

Tempo $O(n \log n)$ · memória $O(n)$

EXEMPLO DE CÓDIGO

solucao.py

```
1 quantidade_de_cenarios = int(input())
2
3 for numero_do_cenario in range(1, quantidade_de_cenarios + 1):
4     quantidade_de_renas, quantidade_escolhida = map(int, input().split())
5     renas = []
6
7     for _ in range(quantidade_de_renas):
8         nome, peso, idade, altura = input().split()
9
10        # Cada tupla funciona como um registro: os quatro campos da rena
11        # permanecem juntos durante toda a ordenação.
12        renas.append((nome, int(peso), int(idade), float(altura)))
13
14    # As tuplas-chave são comparadas campo a campo:
15    # peso decrescente, idade crescente, altura crescente e nome crescente.
16    renas.sort(
17        key=lambda rena: (
18            -renas[1],
19            rena[2],
20            rena[3],
21            rena[0],
22        )
23    )
24
25    print(f"CENARIO {{{numero_do_cenario}}}")
26
27    # A lista já está no ranking correto; basta imprimir o prefixo solicitado.
28    for posicao, rena in enumerate(renas[:quantidade_escolhida], start=1):
29        print(f"{posicao} - {renas[0]}")
```

TÉCNICA

A chave é uma tupla comparada em cascata. O sinal negativo transforma apenas o peso em ordem decrescente.

ARMADILHA

`reverse=True` inverteria todos os critérios. A tupla permite misturar sentidos e desempates corretamente.

Busca binária da primeira ocorrência

Programa completo que ordena mármore uma vez e usa `bisect_left` em cada consulta.

Entrada: vários casos com valores e consultas. Saída: primeira...

$O(n \log n + q \log n)$ por caso

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 # bisect_left faz busca binária e devolve a primeira posição possível.
2 from bisect import bisect_left
3
4
5 # O contador produz o cabeçalho CASE# pedido pelo enunciado.
6 caso = 0
7
8 while True:
9     n, q = map(int, input().split())
10
11     # O par 0 0 encerra a entrada.
12     if n == 0 and q == 0:
13         break
14
15     caso += 1
16
17     valores = []
18
19     for _ in range(n):
20         valor = int(input())
21         valores.append(valor)
22
23     # A ordenação é o pré-processamento necessário para todas as consultas.
24     valores.sort()
25
26     print(f"CASE# {caso}:")
27
28     for _ in range(q):
29         valor = int(input())
30
31         # Procuramos a primeira ocorrência, não apenas qualquer ocorrência.
32         pos = bisect_left(valores, valor)
33
34         # O ponto de inserção ainda precisa ser validado.
35         if pos < len(valores) and valores[pos] == valor:
36             print(f"{valor} found at {pos + 1}")
37         else:
38             print(f"{valor} not found")
```

TÉCNICA

`bisect_left` devolve o ponto mais à esquerda onde o alvo poderia entrar. O código ainda confirma se ele existe.

ARMADILHA

A busca depende da ordenação. A posição interna começa em 0, mas o enunciado imprime começando em 1.

Pilha: balanço de parênteses até EOF

Programa completo do beecrowd 1068. A pilha representa aberturas que ainda aguardam fechamento.

Entrada: expressões até EOF. Saída: correct ou incorrect.

Tempo O(n) · memória O(n) por expressão

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 import sys
2
3
4 def esta_balançada(expressao):
5     """Verifica se os parênteses de uma expressão abrem e fecham na ordem correta."""
6     pilha = []
7
8     for caractere in expressao:
9         if caractere == "(":
10             # A abertura fica pendente até aparecer o fechamento correspondente.
11             pilha.append(caractere)
12         elif caractere == ")":
13             # Fechar sem nenhuma abertura disponível invalida a expressão.
14             if not pilha:
15                 return False
16             pilha.pop()
17
18     # Se ainda há itens na pilha, sobraram aberturas sem fechamento.
19     return not pilha
20
21
22 # Cada linha da entrada é uma expressão; a leitura termina no fim do arquivo.
23 for linha in sys.stdin:
24     expressao = linha.rstrip("\n")
25     print("correct" if esta_balançada(expressao) else "incorrect")
```

TÉCNICA

Ao ler '(', empilhe. Ao ler ')', precisa existir uma abertura disponível. No final, a pilha deve estar vazia.

ARMADILHA

Contagens iguais não garantem ordem correta. ')' deve ser rejeitada imediatamente.

Fila com deque: jogando cartas fora

Programa completo do beecrowd 1110. O topo fica no front e a base no back do deque.

Entrada: valores n até sentinela 0. Saída: descartes e restante.

Tempo $O(n)$ · memória $O(n)$ por caso

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 from collections import deque
2
3
4 while True:
5     quantidade = int(input())
6
7     # O valor zero não representa um baralho: ele encerra a entrada.
8     if quantidade == 0:
9         break
10
11     # O início do deque é o topo do baralho e o fim é a base.
12     cartas = deque(range(1, quantidade + 1))
13     descartadas = []
14
15     while len(cartas) > 1:
16         # A carta do topo sai definitivamente do baralho.
17         descartadas.append(cartas.popleft())
18
19         # A nova carta do topo é retirada e recolocada na base.
20         cartas.append(cartas.popleft())
21
22     texto_descartadas = ", ".join(map(str, descartadas))
23     print(f"Discarded cards: {texto_descartadas}")
24     print(f"Remaining card: {cartas[0]}")
```

TÉCNICA

popleft descarta o topo; append(popleft()) move o novo topo para a base. Cada operação nas pontas custa $O(1)$.

ARMADILHA

list.pop(0) desloca os demais elementos e custa $O(n)$. Para fila, prefira collections.deque.

Pilha, fila e heap simulados em paralelo

Programa completo do beecrowd 1340. Cada remoção elimina as estruturas que não poderiam devolver o valor observado.

Entrada: casos até EOF com operações 1 x e 2 x. Saída: classif...

Tempo $O(n \log n)$ · memória $O(n)$

EXEMPLO DE CÓDIGO

solucao.py

```
1 from collections import deque
2 import heapq
3 import sys
4
5
6 def classificar(operacoes):
7     """Simula as três estruturas e classifica quais reproduzem as remoções."""
8     pilha = []
9     fila = deque()
10    prioridade = [] # heapq é min-heap; valores negativos simulam uma max-heap.
11
12    pode_ser_pilha = True
13    pode_ser_fila = True
14    pode_ser_prioridade = True
15
16    for tipo, valor in operacoes:
17        if tipo == 1:
18            # Toda inserção é reproduzida nas candidatas que ainda são possíveis.
19            if pode_ser_pilha:
20                pilha.append(valor)
21            if pode_ser_fila:
22                fila.append(valor)
23            if pode_ser_prioridade:
24                heapq.heappush(prioridade, -valor)
25            continue
26
27        # Na remoção, o valor observado precisa ser exatamente o que a estrutura
28        # candidata retiraria naquele momento.
29        if pode_ser_pilha and (not pilha or pilha.pop() != valor):
30            pode_ser_pilha = False
31        if pode_ser_fila and (not fila or fila.popleft() != valor):
32            pode_ser_fila = False
33        if pode_ser_prioridade and (
34            not prioridade or -heapq.heappop(prioridade) != valor
35        ):
36            pode_ser_prioridade = False
37
38    candidatas = sum((pode_ser_pilha, pode_ser_fila, pode_ser_prioridade))
39    if candidatas == 0:
40        return "impossible"
41    if candidatas > 1:
42        return "not sure"
43    if pode_ser_pilha:
44        return "stack"
45    if pode_ser_fila:
46        return "queue"
47    return "priority queue"
48
49
50 # Os casos aparecem até EOF. Usar um iterador facilita consumir exatamente
51 # a quantidade de operações declarada em cada caso.
52 dados = iter(map(int, sys.stdin.buffer.read().split()))
53
54 while True:
55     try:
56         quantidade = next(dados)
57     except StopIteration:
58         break
59
60    operacoes = [(next(dados), next(dados)) for _ in range(quantidade)]
61    print(classificar(operacoes))
```

TÉCNICA

A mesma inserção é reproduzida nas três candidatas. Uma remoção compara LIFO, FIFO e maior prioridade.

ARMADILHA

heapq é min-heap. Valores negativos simulam a retirada do maior elemento.

Transformação de string em três fases

Programa completo do beecrowd 1024. Cada regra recebe o resultado da fase anterior.

Entrada: quantidade e mensagens completas. Saída: mensagem cri...

Tempo O(n) · memória O(n) por mensagem

EXEMPLO DE CÓDIGO

solucao.py

```
1 import sys
2
3
4 def criptografar(mensagem):
5     """Executa, na ordem, as três fases descritas pelo enunciado."""
6     # Fase 1: somente letras ASCII avançam três códigos.
7     deslocada = [
8         chr(ord(caractere) + 3)
9         if "A" <= caractere <= "Z" or "a" <= caractere <= "z"
10        else caractere
11        for caractere in mensagem
12    ]
13    # Fase 2: a mensagem inteira é invertida.
14    invertida = deslocada[::-1]
15    metade = len(invertida) // 2
16    # Fase 3: os caracteres da metade final recuam um código.
17    return "".join(
18        invertida[:metade]
19        + [chr(ord(caractere) - 1) for caractere in invertida[metade:]]
20    )
21
22
23 def main():
24     # A primeira linha informa quantas mensagens devem ser transformadas.
25     linhas = sys.stdin.buffer.read().decode().splitlines()
26     quantidade = int(linhas[0])
27     respostas = [criptografar(linhas[i]) for i in range(1, quantidade + 1)]
28     # write não acrescenta quebra automaticamente; fechamos também a última linha.
29     sys.stdout.write("\n".join(respostas) + "\n")
30
31
32 if __name__ == "__main__":
33     main()
```

TÉCNICA

ord e chr deslocam códigos; [::-1] inverte; len // 2 define a metade após a inversão.

ARMADILHA

Executar fases fora de ordem muda as posições usadas pela terceira regra.

Máquina de estados em uma frase

Programa completo do beecrowd 1234. O booleano informa como a próxima letra deve ser escrita.

Entrada: sentenças até EOF. Saída: maiúsculas e minúsculas alt...

Tempo O(n) · memória O(n) por linha

EXEMPLO DE CÓDIGO

solucao.py

```
1 import sys
2
3
4 def transformar_sentenca(sentenca):
5     """Alterna a caixa das letras sem deixar espaços alterarem o estado."""
6     usar_maiuscula = True
7     resposta = []
8
9     for caractere in sentenca:
10         if caractere.isalpha():
11             # O estado determina a caixa desta letra e muda para a próxima.
12             resposta.append(
13                 caractere.upper() if usar_maiuscula else caractere.lower()
14             )
15             usar_maiuscula = not usar_maiuscula
16         else:
17             # Espaços e sinais são copiados sem consumir a alternância.
18             resposta.append(caractere)
19
20     return "".join(resposta)
21
22
23 def main():
24     # O problema fornece sentenças até o fim do arquivo.
25     linhas = sys.stdin.buffer.read().decode().splitlines()
26     sys.stdout.write("\n".join(map(transformar_sentenca, linhas)) + "\n")
27
28
29 if __name__ == "__main__":
30     main()
```

TÉCNICA

O estado troca somente depois de uma letra. Espaços e sinais são copiados sem consumir a alternância.

ARMADILHA

Usar o índice absoluto falha quando espaços não devem contar.

Intercalação com sobra da maior string

Programa completo do beecrowd 1238. Intercala o prefixo comum e anexa o restante sem perder caracteres.

Entrada: T pares de strings. Saída: uma combinação por linha.

Tempo $O(|a|+|b|)$ · memória $O(|a|+|b|)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 import sys
2
3
4 def combinar(a, b):
5     """Intercala o prefixo comum e acrescenta a sobra da string maior."""
6     # Intercalamos enquanto as duas strings possuem a posição i.
7     limite = min(len(a), len(b))
8     intercalada = "".join(
9         a[i] + b[i] for i in range(limite)
10    )
11
12    # Apenas uma das duas sobras será não vazia.
13    return intercalada + a[limite:] + b[limite:]
14
15
16 def main():
17     # A primeira linha informa quantos pares devem ser combinados.
18     linhas = sys.stdin.buffer.read().decode().splitlines()
19     quantidade = int(linhas[0])
20     respostas = []
21
22     for i in range(1, quantidade + 1):
23         a, b = linhas[i].split()
24         respostas.append(combinar(a, b))
25
26     # Uma única escrita costuma ser mais eficiente do que vários prints.
27     sys.stdout.write("\n".join(respostas) + "\n")
28
29
30 if __name__ == "__main__":
31     main()
```

TÉCNICA

`min(len(a), len(b))` define quantos pares existem. Depois do limite, somente uma das sobras é não vazia.

ARMADILHA

`zip(a, b)` puro descarta a sobra da string maior. Ela precisa ser anexada explicitamente.

DP da maior substring comum

Programa completo do beecrowd 1237. Coincidências estendem a diagonal anterior; divergências valem zero.

Entrada: pares de linhas até EOF. Saída: tamanho do maior trec...

Tempo $O(n \cdot m)$ · memória $O(\min(n, m))$

EXEMPLO DE CÓDIGO

solucao.py

```
1 import sys
2
3
4 def maior_substring_comum(a, b):
5     """Calcula a maior substring comum com DP de memória otimizada."""
6     # b será a menor string para reduzir a memória para  $O(\min(n, m))$ .
7     if len(b) > len(a):
8         a, b = b, a
9
10    # anterior representa a linha anterior da tabela de programação dinâmica.
11    anterior = [0] * (len(b) + 1)
12    melhor = 0
13
14    for caractere_a in a:
15        atual = [0] * (len(b) + 1)
16        for j, caractere_b in enumerate(b, start=1):
17            if caractere_a == caractere_b:
18                # A coincidência estende a diagonal; diferenças permanecem zero
19                # porque uma substring não pode pular caracteres.
20                atual[j] = anterior[j - 1] + 1
21                melhor = max(melhor, atual[j])
22        anterior = atual
23
24    return melhor
25
26
27 def main():
28     # Cada caso possui duas linhas e a entrada termina em EOF.
29     linhas = sys.stdin.buffer.read().decode().splitlines()
30     respostas = []
31     for i in range(0, len(linhas) - 1, 2):
32         respostas.append(str(maior_substring_comum(linhas[i], linhas[i + 1])))
33     sys.stdout.write("\n".join(respostas) + "\n")
34
35
36 if __name__ == "__main__":
37     main()
```

TÉCNICA

`anterior[j-1]` representa o sufixo comum que terminava nas duas posições anteriores.

ARMADILHA

Substring é contínua; subsequência permite saltos e resolve outro problema.

DFS e árvore de descoberta

Programa completo do beecrowd 1076. A DFS conta arestas que alcançam vértices novos e dobra para ida e volta.

Entrada: T labirintos. Saída: movimentos mínimos de caneta.

Tempo $O(V+E)$ · memória $O(V+E)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 def dfs(vertice, grafo, visitado):
2     """Visita o componente e devolve quantas arestas descobriram vértices novos."""
3     visitado[vertice] = True
4     arestas_da_busca = 0
5
6     # A DFS segue cada vizinho ainda não visitado e aprofunda a busca.
7     for vizinho in grafo[vertice]:
8         if not visitado[vizinho]:
9             # Esta aresta entra na árvore da DFS porque alcançou um vértice novo.
10            arestas_da_busca += 1
11            arestas_da_busca += dfs(vizinho, grafo, visitado)
12
13    return arestas_da_busca
14
15
16 # O primeiro valor informa quantos labirintos serão processados.
17 quantidade_de_casos = int(input())
18
19 for _ in range(quantidade_de_casos):
20     origem = int(input())
21     vertices, arestas = map(int, input().split())
22
23     # Cada posição guarda somente os vizinhos do respectivo vértice.
24     grafo = [[] for _ in range(vertices)]
25
26     for _ in range(arestas):
27         a, b = map(int, input().split())
28         # O labirinto é um grafo não direcionado, então registramos os dois sentidos.
29         grafo[a].append(b)
30         grafo[b].append(a)
31
32     visitado = [False] * vertices
33     arestas_da_busca = dfs(origem, grafo, visitado)
34
35     # Para sair da origem e voltar a ela, cada aresta da árvore é percorrida
36     # uma vez na ida e outra na volta.
37     print(arestas_da_busca * 2)
```

TÉCNICA

A lista de adjacência guarda vizinhos. visitado evita repetir vértices e neutraliza arestas duplicadas.

ARMADILHA

Em grafo não direcionado, registre a e b nos dois sentidos. Em direcionado, não invente a volta.

BFS no tabuleiro do cavalo

Programa completo do beecrowd 1100. A fila visita casas em camadas de distância crescente.

Entrada: pares de casas até EOF. Saída: menor número de movime...

O(64·8) por caso · memória O(64)

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 from collections import deque
2 import sys
3
4
5 # Os oito deslocamentos possíveis de um cavalo: duas casas em um eixo
6 # e uma casa no outro eixo.
7 MOVIMENTOS = (
8     (2, 1), (2, -1), (-2, 1), (-2, -1),
9     (1, 2), (1, -2), (-1, 2), (-1, -2),
10 )
11
12
13 def para_coordenada(posicao):
14     """Converte uma casa como 'a1' para índices de linha e coluna entre 0 e 7."""
15     coluna = ord(posicao[0]) - ord("a")
16     linha = int(posicao[1]) - 1
17     return linha, coluna
18
19
20 def esta_no_tabuleiro(linha, coluna):
21     """Informa se a coordenada pertence ao tabuleiro 8 x 8."""
22     return 0 <= linha < 8 and 0 <= coluna < 8
23
24
25 def bfs(origem, destino):
26     """Calcula o menor número de movimentos entre duas casas com BFS."""
27     inicio = para_coordenada(origem)
28     fim = para_coordenada(destino)
29
30     # A fila mantém as casas por camadas de distância.
31     fila = deque([(inicio[0], inicio[1], 0)])
32     visitado = [[False] * 8 for _ in range(8)]
33     visitado[inicio[0]][inicio[1]] = True
34
35     while fila:
36         linha, coluna, distancia = fila.popleft()
37
38         # A primeira retirada do destino ocorre pelo menor caminho possível.
39         if (linha, coluna) == fim:
40             return distancia
41
42         for delta_linha, delta_coluna in MOVIMENTOS:
43             nova_linha = linha + delta_linha
44             nova_coluna = coluna + delta_coluna
45
46             if (
47                 esta_no_tabuleiro(nova_linha, nova_coluna)
48                 and not visitado[nova_linha][nova_coluna]
49             ):
50                 # Marcar ao entrar na fila impede que a mesma casa seja enfileirada
51                 # várias vezes por caminhos diferentes.
52                 visitado[nova_linha][nova_coluna] = True
53                 fila.append((nova_linha, nova_coluna, distancia + 1))
54
55     return -1 # Não ocorre em um tabuleiro comum, mas completa o contrato da função.
56
57
58 # O problema fornece pares de casas até o fim do arquivo.
59 for linha_de_entrada in sys.stdin:
60     linha_de_entrada = linha_de_entrada.strip()
61     if not linha_de_entrada:
62         continue
63
64     origem, destino = linha_de_entrada.split()
65     resposta = bfs(origem, destino)
66     print(f"To get from {origem} to {destino} takes {resposta} knight moves.")
```

TÉCNICA

Os oito deslocamentos formam as arestas do grafo. A primeira retirada do destino tem a menor distância.

ARMADILHA

Marque a casa quando ela entra na fila, não quando sai, para evitar inserções duplicadas.

Dijkstra com lista e fila de prioridade

Programa completo para menor caminho com pesos não negativos, usando heap e descarte de entradas antigas.

Entrada: grafo direcionado e uma origem. Saída: distâncias ou -1.

$O((V+E) \log V)$ · memória $O(V+E)$

EXEMPLO DE CÓDIGO

solucao.py

```
1 import heapq
2 import sys
3
4
5 def dijkstra(grafo, origem):
6     infinito = 10**18
7     distancia = [infinito] * len(grafo)
8     distancia[origem] = 0
9     fila = [(0, origem)]
10
11     while fila:
12         custo_atual, vertice = heapq.heappop(fila)
13
14         # Uma rota melhor já foi inserida depois desta.
15         if custo_atual != distancia[vertice]:
16             continue
17
18         for vizinho, peso in grafo[vertice]:
19             novo_custo = custo_atual + peso
20
21             # Relaxamento: melhora a estimativa do vizinho.
22             if novo_custo < distancia[vizinho]:
23                 distancia[vizinho] = novo_custo
24                 heapq.heappush(fila, (novo_custo, vizinho))
25
26     return distancia
27
28
29 def main():
30     vertices, arestas, origem = map(int, input().split())
31     grafo = [[] for _ in range(vertices)]
32
33     for _ in range(arestas):
34         a, b, peso = map(int, input().split())
35         grafo[a].append((b, peso))
36
37     resposta = dijkstra(grafo, origem)
38     print*(-1 if d == 10**18 else d for d in resposta)
39
40
41 if __name__ == "__main__":
42     main()
```

TÉCNICA

Relaxar é testar se passar pelo vértice atual cria custo menor. A heap sempre oferece a menor estimativa.

ARMADILHA

Dijkstra não funciona com pesos negativos. BFS minimiza quantidade de arestas, não a soma dos pesos.

Círculos: contenção e separação

Programa completo com dois tipos de consulta: círculo dentro de outro e dois círculos em um retângulo.

Entrada: T consultas DENTRO ou RETANGULO. Saída: S ou N.

Tempo $O(T)$ · memória $O(1)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 def circulo_dentro(r1, x1, y1, r2, x2, y2):
2     # Um círculo maior nunca cabe no menor.
3     if r2 > r1:
4         return False
5
6     dx = x1 - x2
7     dy = y1 - y2
8     limite = r1 - r2
9
10    # Distância entre centros <= diferença dos raios.
11    return dx * dx + dy * dy <= limite * limite
12
13
14 def cabem_no_retangulo(largura, altura, r1, r2):
15     # Cada diâmetro precisa caber isoladamente.
16     for raio in (r1, r2):
17         if 2 * raio > largura or 2 * raio > altura:
18             return False
19
20     # Cantos opostos maximizam a distância.
21     dx = largura - r1 - r2
22     dy = altura - r1 - r2
23     return dx * dx + dy * dy >= (r1 + r2) ** 2
24
25
26 def main():
27     t = int(input())
28
29     for _ in range(t):
30         partes = input().split()
31         tipo = partes[0]
32         valores = list(map(float, partes[1:]))
33
34         if tipo == "DENTRO":
35             resposta = circulo_dentro(*valores)
36         else:
37             resposta = cabem_no_retangulo(*valores)
38
39         print("S" if resposta else "N")
40
41
42 if __name__ == "__main__":
43     main()
```

TÉCNICA

Contenção usa diferença dos raios; não sobreposição usa soma. Comparar quadrados evita sqrt.

ARMADILHA

Antes de elevar a diferença ao quadrado, teste qual raio é maior. Tangência usa $\leq$ ou $\geq$.

Distância de ponto a segmento

Programa completo que projeta cada ponto no segmento e imprime a menor distância com duas casas.

Entrada: segmento e Q pontos. Saída: uma distância por ponto.

Tempo $O(Q)$ · memória $O(1)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 import math
2
3
4 def distancia_ponto_segmento(p, a, b):
5     px, py = p
6     ax, ay = a
7     bx, by = b
8     vx = bx - ax
9     vy = by - ay
10    comprimento2 = vx * vx + vy * vy
11
12    # Segmento degenerado: A e B são o mesmo ponto.
13    if comprimento2 == 0.0:
14        return math.hypot(px - ax, py - ay)
15
16    # Projeção na reta, depois limitada ao segmento.
17    t = ((px - ax) * vx + (py - ay) * vy) / comprimento2
18    t = max(0.0, min(1.0, t))
19
20    qx = ax + t * vx
21    qy = ay + t * vy
22    return math.hypot(px - qx, py - qy)
23
24
25 def main():
26     ax, ay, bx, by = map(float, input().split())
27     consultas = int(input())
28     a = (ax, ay)
29     b = (bx, by)
30
31     for _ in range(consultas):
32         ponto = tuple(map(float, input().split()))
33         distancia = distancia_ponto_segmento(ponto, a, b)
34         print(f"{distancia:.2f}")
35
36
37 if __name__ == "__main__":
38     main()
```

TÉCNICA

t entre 0 e 1 usa a perpendicular; fora desse intervalo, a extremidade mais próxima vence.

ARMADILHA

Distância à reta infinita pode cair fora do segmento e produzir resposta impossível.

Circuncírculo e comparação com EPS

Programa completo que testa se todos os pontos pertencem ao círculo definido pelos três primeiros.

Entrada: N pontos. Saída: SIM, NAO ou COLINEARES.

Tempo O(N) · memória O(N)

EXEMPLO DE CÓDIGO

solucao.py

```
1 import math
2
3
4 EPS = 1e-7
5
6
7 def circuncirculo(a, b, c):
8     ax, ay = a
9     bx, by = b
10    cx, cy = c
11    d = 2.0 * (
12        ax * (by - cy)
13        + bx * (cy - ay)
14        + cx * (ay - by)
15    )
16
17    # Determinante zero: os três pontos são colineares.
18    if math.isclose(d, 0.0, abs_tol=EPS):
19        return None
20
21    a2 = ax * ax + ay * ay
22    b2 = bx * bx + by * by
23    c2 = cx * cx + cy * cy
24    ux = (a2 * (by - cy) + b2 * (cy - ay)
25        + c2 * (ay - by)) / d
26    uy = (a2 * (cx - bx) + b2 * (ax - cx)
27        + c2 * (bx - ax)) / d
28    raio2 = (ax - ux) ** 2 + (ay - uy) ** 2
29    return ux, uy, raio2
30
31
32 def main():
33     n = int(input())
34     pontos = [tuple(map(float, input().split())) for _ in range(n)]
35     circulo = circuncirculo(*pontos[:3])
36
37     if circulo is None:
38         print("COLINEARES")
39         return
40
41     ux, uy, raio2 = circulo
42     todos = all(
43         math.isclose(
44             (x - ux) ** 2 + (y - uy) ** 2,
45             raio2, rel_tol=EPS, abs_tol=EPS
46         )
47         for x, y in pontos
48     )
49     print("SIM" if todos else "NAO")
50
51
52 if __name__ == "__main__":
53     main()
```

TÉCNICA

Três pontos não colineares determinam um círculo. O determinante detecta a degeneração.

ARMADILHA

Resultados de divisões não devem ser comparados com ==. Use tolerância relativa e absoluta.

Áreas, seno e conversão para radianos

Programa completo que atende consultas de áreas do quadrado com arcos e da razão pentágono-quadrado.

Entrada: T comandos AREA ou PENTAGONO. Saída: valores formatad...

Tempo O(T) · memória O(1)

EXEMPLO DE CÓDIGO

solucao.py

```
1 import math
2
3
4 def areas(lado):
5     quadrado = lado * lado
6     complemento = quadrado - math.pi * quadrado / 4.0
7     segmento = (
8         quadrado * (4 * math.pi - 3 * math.sqrt(3)) / 24
9     )
10    a3 = 8 * segmento + 8 * complemento - 4 * quadrado
11    a2 = 4 * complemento - 2 * a3
12    a1 = quadrado - a2 - a3
13    return a1, a2, a3
14
15
16 def lado_do_quadrado(lado_do_pentagono):
17     # math.sin recebe radianos, não graus.
18     return (
19         lado_do_pentagono
20         * math.sin(math.radians(108))
21         / math.sin(math.radians(63))
22     )
23
24
25 def main():
26     t = int(input())
27
28     for _ in range(t):
29         tipo, texto = input().split()
30         valor = float(texto)
31
32         if tipo == "AREA":
33             a1, a2, a3 = areas(valor)
34             print(f"{a1:.3f} {a2:.3f} {a3:.3f}")
35         else:
36             print(f"{lado_do_quadrado(valor):.10f}")
37
38
39 if __name__ == "__main__":
40     main()
```

TÉCNICA

Fórmulas constantes viram funções O(1). math.radians faz a conversão obrigatória antes de sin.

ARMADILHA

A ordem das áreas e a quantidade de casas pertencem ao enunciado. Não arredonde valores intermediários.

Backtracking com poda por limite

Programa completo que maximiza uma soma sem ultrapassar o limite, escolhendo, explorando e desfazendo.

Entrada: itens positivos e capacidade. Saída: maior soma possível.

Pior caso $O(2^n)$ · podas reduzem a busca

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 def melhor_soma(valores, limite):
2     n = len(valores)
3     sufixo = [0] * (n + 1)
4
5     # Bound otimista: soma de tudo que ainda resta.
6     for i in range(n - 1, -1, -1):
7         sufixo[i] = sufixo[i + 1] + valores[i]
8
9     melhor = 0
10
11     def buscar(i, soma):
12         nonlocal melhor
13
14         if soma > limite:
15             return # ramo inviável
16
17         melhor = max(melhor, soma)
18
19         if i == n:
20             return
21
22         # Mesmo pegando tudo, este ramo não supera o incumbente.
23         if soma + sufixo[i] <= melhor:
24             return
25
26         buscar(i + 1, soma + valores[i]) # escolher
27         buscar(i + 1, soma)             # não escolher
28
29     buscar(0, 0)
30     return melhor
31
32
33 def main():
34     n, limite = map(int, input().split())
35     valores = list(map(int, input().split()))
36     assert len(valores) == n
37     print(melhor_soma(valores, limite))
38
39
40 if __name__ == "__main__":
41     main()
```

TÉCNICA

Backtracking explora escolhas; branch and bound poda um ramo que não pode superar a melhor solução já encontrada.

ARMADILHA

O bound deve ser otimista e seguro. Um limite que subestima o potencial pode eliminar a resposta ótima.

Guloso com pilha monotônica

Programa completo do beecrowd 1084. Dígitos maiores removem dígitos menores anteriores.

Entrada: n, m e número até sentinela 0 0. Saída: maior número.

Tempo $O(n)$ · memória $O(n)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 import sys
2
3 # buffer evita o custo de camadas extras de entrada em casos grandes.
4 entrada = sys.stdin.buffer
5
6 while True:
7     n, m = map(int, entrada.readline().split())
8
9     # 0 par 0 0 encerra a entrada.
10    if n == 0 and m == 0:
11        break
12
13    numero = entrada.readline().strip().decode()
14
15    # A pilha mantém o maior prefixo possível em ordem monotônica.
16    pilha = []
17    apagados = 0
18
19    for digito in numero:
20        # Um dígito maior substitui dígitos menores anteriores enquanto ainda
21        # houver remoções disponíveis.
22        while pilha and apagados < m and digito > pilha[-1]:
23            pilha.pop()
24            apagados += 1
25
26        # A resposta precisa terminar com exatamente n - m algarismos.
27        if len(pilha) < n - m:
28            pilha.append(digito)
29
30    print("".join(pilha))
```

TÉCNICA

A primeira posição diferente decide o maior número. Cada dígito entra e sai da pilha no máximo uma vez.

ARMADILHA

Uma estratégia gulosa precisa de justificativa. Escolha local sem prova pode falhar em outro problema.

DP de troco ilimitado

Programa completo do beecrowd 1034. mochila[j] guarda o menor número de blocos para formar j.

Entrada: tipos de bloco e comprimento alvo. Saída: mínimo de b...

Tempo $O(N \cdot M)$ · memória $O(M)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 N, M = 0, 0
2 mochila = []
3 blocos = []
4
5
6 def calculaMochila():
7     # mochila[j] guarda o menor número de blocos que forma exatamente j.
8     mochila = [-1 for _ in range(M + 1)]
9     mochila[0] = 0
10
11     for bloco in blocos:
12         # O percurso crescente permite reutilizar o mesmo tamanho de bloco.
13         for j in range(bloco, M + 1):
14             if(mochila[j - bloco] != -1):
15                 if(mochila[j] == -1):
16                     mochila[j] = mochila[j - bloco] + 1
17                 else:
18                     mochila[j] = min(mochila[j], mochila[j - bloco] + 1)
19
20     return mochila[M]
21
22
23 T = int(input())
24
25 for _ in range(T):
26     N, M = map(int, input().strip().split(' '))
27     # Os tamanhos disponíveis formam as transições da programação dinâmica.
28     blocos = map(int, input().strip().split(' '))
29
30     print(calculaMochila())
```

TÉCNICA

O laço crescente permite reutilizar o bloco atual. O estado j-bloco precisa ser alcançável.

ARMADILHA

Percorrer para trás resolveria a versão em que cada bloco só pode ser usado uma vez.

Mochila 0/1 com memória comprimida

Programa completo do beecrowd 1286. Cada pedido pode ser escolhido no máximo uma vez.

Entrada: pedidos com valor e peso. Saída: melhor valor na capa...

Tempo $O(n \cdot \text{capacidade})$ · memória $O(\text{capacidade})$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 import sys
2
3 # O formato possui muitos inteiros; o buffer oferece leitura direta em bytes.
4 entrada = sys.stdin.buffer
5
6 while True:
7     n = int(entrada.readline())
8
9     if n == 0:
10         break
11
12     pizzas = int(entrada.readline())
13
14     # dp[c] é o maior tempo economizado usando, no máximo, c pizzas.
15     dp = [0] * (pizzas + 1)
16
17     for _ in range(n):
18         tempo, quantidade = map(int, entrada.readline().split())
19
20         # Percorre de trás para frente porque cada pedido só pode ser escolhido
21         # uma vez; avançar permitiria reutilizar o pedido na mesma rodada.
22         for capacidade in range(pizzas, quantidade - 1, -1):
23             dp[capacidade] = max(
24                 dp[capacidade],
25                 dp[capacidade - quantidade] + tempo
26             )
27
28     print(f"{dp[pizzas]} min.")
```

TÉCNICA

A capacidade é percorrida de trás para frente; assim o item atual não alimenta outro estado da mesma rodada.

ARMADILHA

A direção do laço define 0/1 ou ilimitada. A fórmula isolada não revela essa diferença.

Recursão e memoização de dois estados

Programa completo do beecrowd 1029. O cache guarda Fibonacci e a quantidade teórica de chamadas.

Entrada: consultas n. Saída: fib(n) e chamadas recursivas.

Tempo $O(\text{maior } n)$ · memória $O(\text{maior } n)$

EXEMPLO DE CÓDIGO

○ ○ ○

solucao.py

```
1 # F memoriza Fibonacci; CF memoriza o total de nós da árvore de chamadas.
2 F = [-1 for _ in range(40)]
3 CF = [-1 for _ in range(40)]
4
5 F[0] = 0
6 F[1] = 1
7
8 CF[0] = 1
9 CF[1] = 1
10
11 def calcula(n):
12     # O estado só é expandido na primeira vez em que aparece.
13     if(F[n] == -1):
14         result1, num_calls1 = calcula(n - 1)
15         result2, num_calls2 = calcula(n - 2)
16         F[n] = result1 + result2
17         # As duas subárvores mais 1 para a chamada calcula(n).
18         CF[n] = num_calls1 + num_calls2 + 1
19     return (F[n], CF[n])
20
21 # A memória permanece entre os casos e reaproveita resultados anteriores.
22 N = int(input())
23
24 for _ in range(N):
25     X = int(input())
26     result, num_calls = calcula(X)
27     # O enunciado não conta a chamada inicial; por isso subtraímos 1.
28     print(f'fib({X}) = {num_calls - 1} calls = {result}')
```

TÉCNICA

F[n] e CF[n] são estados distintos da mesma recorrência. Cada índice é calculado apenas uma vez.

ARMADILHA

Memoização evita recomputação, mas a versão top-down ainda usa pilha. A chave deve identificar todo o estado.